

[Algorithm]

탐욕 알고리즘(Greedy Algorithm)

인하대학교 소프트웨어융합공학과

김종훈 교수

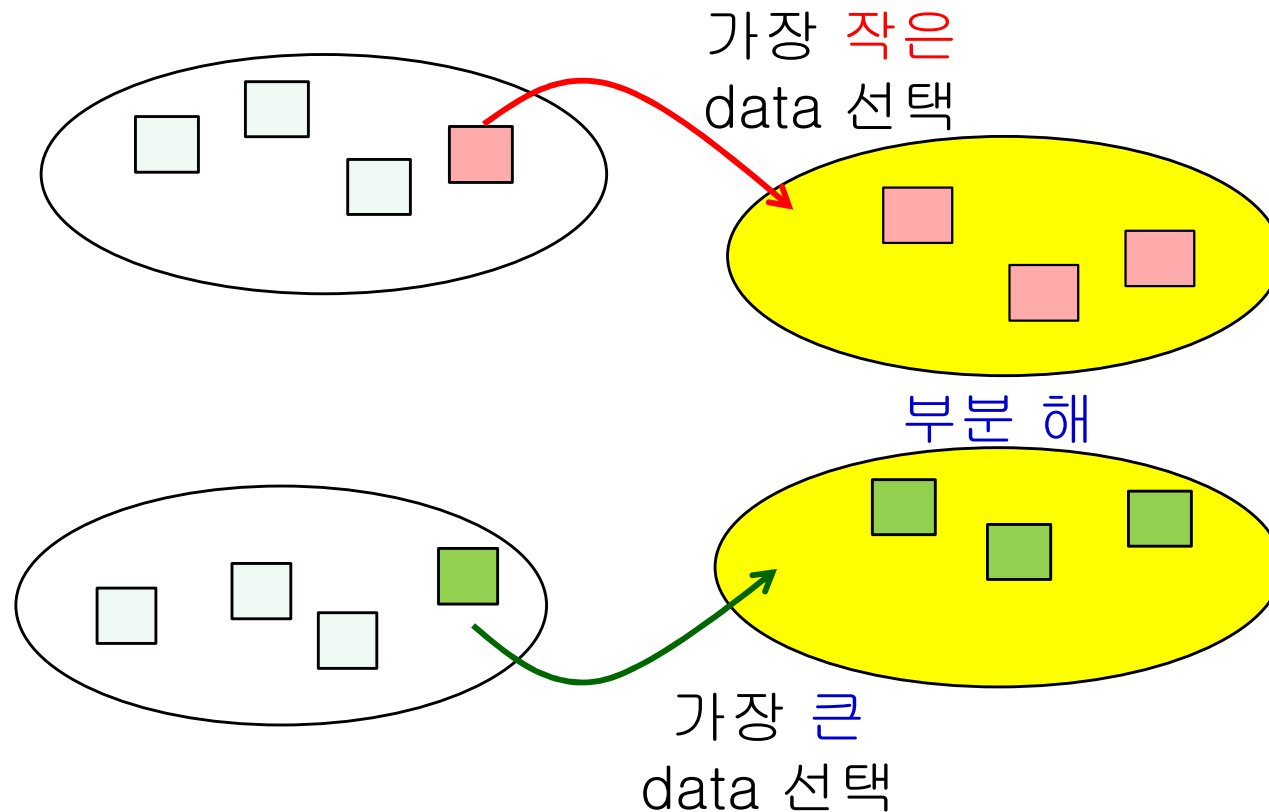


그리디 (Greedy) 알고리즘

- 그리디 알고리즘은 최적화 문제를 해결하는 알고리즘
 - 최적화 (optimization) 문제
 - 가능한 해들 중에서 가장 좋은 (최대 또는 최소) 해를 찾는 문제
- 욕심쟁이 방법, 탐욕적 방법, 탐욕 알고리즘 등으로 불림
- 그리디 알고리즘은 (입력) 데이터 간의 관계를 고려하지 않고 수행 과정에서 '욕심내어' 최소값 또는 최대값을 가진 데이터를 선택
 - 이러한 선택을 '근시안적'인 선택이라고 말하기도 함

그리디 (Greedy) 알고리즘

- 그리디 알고리즘은 근시안적인 선택으로 부분적인 최적해를 찾고, 이들을 모아서 문제의 최적해를 얻는다.



그리디 알고리즘 수행 과정

그리디 (Greedy) 알고리즘

- 그리디 알고리즘은 일단 한 번 선택하면, 이를 절대로 반복하지 않는다.
 - 즉, 선택한 데이터를 버리고 다른 것을 취하지 않는다.
- 이러한 특성 때문에 대부분의 그리디 알고리즘들은 매우 단순하며, 또한 제한적인 문제들만이 그리디 알고리즘으로 해결된다.

그리디 (Greedy) 알고리즘

□ 수업 시간표 짜기 문제

– 탐욕 알고리즘 (Greedy Algorithm)

- 각 단계에서 최적의 수를 찾아 전역 최적해 구하기

– Ex) 시간이 일부 겹치는 과목들 중 되도록 많은 수업을 들으려는 경우

- 가장 빨리 끝나는 과목을 골라 먼저 신청
- 첫 번째 과목이 끝난 후 시작하여 가장 빨리 끝나는 과목 신청
- 위와 같이 반복

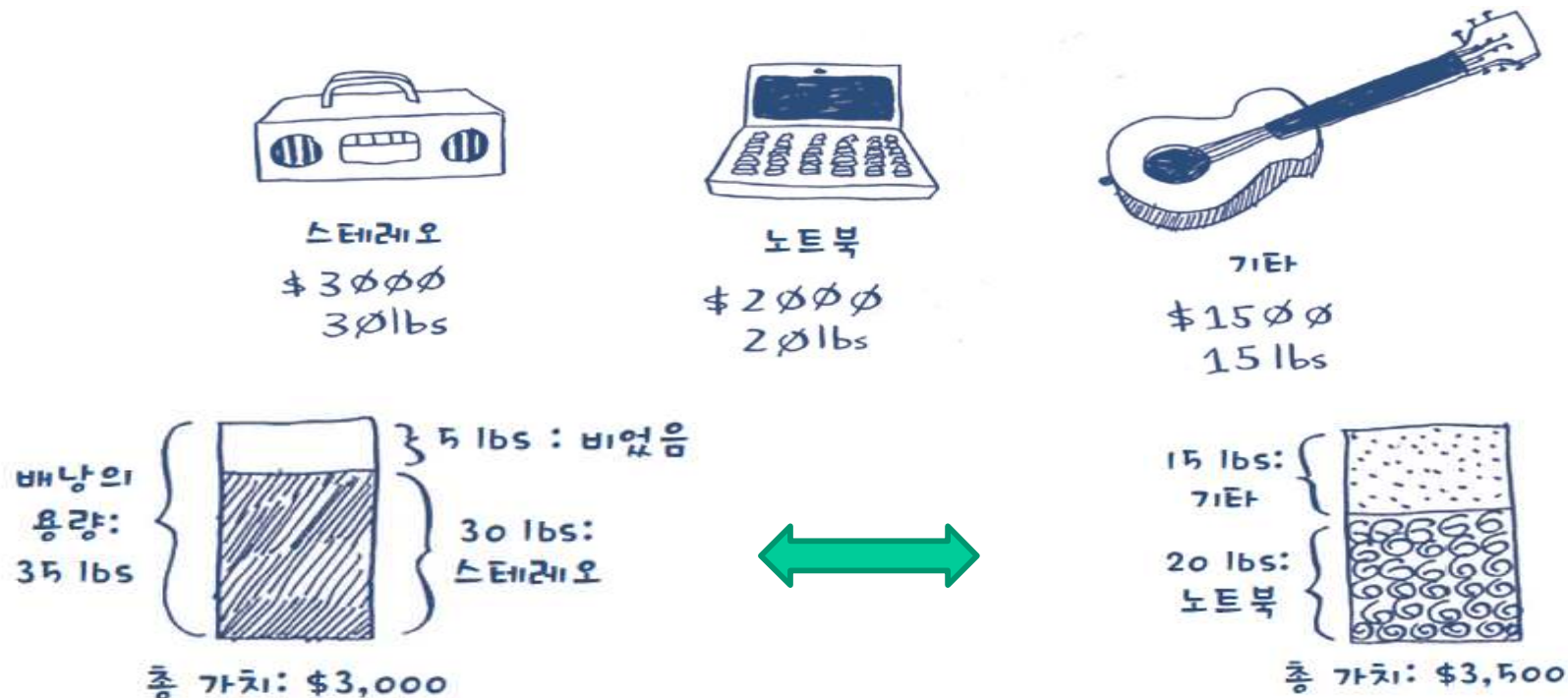


미술	9AM	10AM	✓
영어	9:30AM	10:30AM	X
수학	10AM	11AM	✓
컴퓨터	10:30AM	11:30AM	X
음악	11AM	12PM	✓

그리디 (Greedy) 알고리즘

□ 배낭 채우기 문제

- Ex) 일정 무게만 담을 수 있는 배낭에 최대 가격의 물건들 넣기
 - 가방에 들어갈 수 있는 것 중 가장 비싼 물건을 고름
 - 그 다음으로 들어갈 수 있는 것 중 가장 비싼 것을 고름
 - 위와 같이 반복



1. 부분 배낭 문제

□ 배낭 (Knapsack) 문제

- n 개의 물건이 있고,
- 각 물건은 무게와 가치를 가지고 있으며,
- 배낭이 한정된 무게의 물건들을 담을 수 있을 때,
- 최대의 가치를 갖도록 배낭에 넣을 물건들을 정하는 문제

□ 부분 배낭 (Fractional Knapsack) 문제

- 물건을 부분적으로 담는 것을 허용
- 그리디 알고리즘으로 해결

□ 0-1 배낭 문제 (0/1 배낭 문제)

- 부분 배낭 문제의 원형으로 물건을 통째로 배낭에 넣어야 한다.
- 동적 계획 알고리즘...

부분 배낭 문제

- 부분 배낭 문제에서는 물건을 부분적으로 배낭에 담을 수 있으므로, 최적해를 위해서 '욕심을 내어' 단위 무게 당 가장 값나가는 물건을 배낭에 넣고, 계속해서 그 다음으로 값나가는 물건을 넣는다.
- 그런데 만일 그 다음으로 값나가는 물건을 '통째로' 배낭에 넣을 수 없게 되면, 배낭에 넣을 수 있을 만큼만 물건을 부분적으로 배낭에 담는다.

부분 배낭 문제 그리디 알고리즘

FractionalKnapsack

입력: n 개의 물건, 각 물건의 무게와 가치, 배낭의 용량 C

출력: 배낭에 담은 물건 리스트 L 과 배낭에 담은 물건 가치의 합 v

1. 각 물건에 대해 단위 무게 당 가치를 계산한다.
2. 물건들을 단위 무게 당 가치를 기준으로 내림차순으로 정렬하고, 정렬된 물건 리스트를 S 라고 하자.
3. $L = \emptyset, w = 0, v = 0$
// L 은 배낭에 담은 물건 리스트, w 는 배낭에 담긴 물건들의 무게의 합, v 는 배낭에 담긴 물건들의 가치의 합
4. S 에서 단위 무게 당 가치가 가장 큰 물건 x 를 가져온다.

부분 배낭 문제 그리디 알고리즘

5. while ((w+x의 무게) $\leq C$) {
6. x를 L에 추가시킨다.
7. w = w+x의 무게
8. v = v+x의 가치
9. x를 S에서 제거한다.
10. S에서 단위 무게 당 가치가 가장 큰 물건 x를 가져온다.
- }
11. if ((C-w) > 0) { // 배낭에 물건을 부분적으로 담을 여유가 있으면
12. 물건 x를 (C-w)만큼만 L에 추가한다.
13. v = v + (C-w)만큼의 x의 가치
- }
14. return L, v

부분 배낭 문제 그리디 알고리즘

□ Line 1~2

- 각 물건의 단위 무게 당 가치를 계산하여, 이를 기준으로 물건들을 내림차순으로 정렬한다.

□ Line 5~10

- while-루프를 통해서 다음으로 단위 무게 당 값나가는 물건을 가져다 배낭에 담고, 만일 가져온 물건을 배낭에 담을 경우 배낭의 용량이 초과되면, (즉, while-루프의 조건이 '거짓'이 되면) 가져온 물건을 '통째로' 담을 수 없게 되어 루프를 종료한다.

□ Line 11

- 현재까지 배낭에 담은 물건들의 무게 w 가 배낭의 용량 C 보다 작으면, (즉, if-조건이 '참'이면) line 12~13에서 해당 물건을 $(C-w)$ 만큼만 배낭에 담고, $(C-w)$ 만큼의 x 의 가치를 증가시킨다.

□ Line 14

- 최종적으로 배낭에 담긴 물건들의 리스트 L 과 배낭에 담긴 물건들의 가치 합 v 를 리턴한다.

Fractional Knapsack 알고리즘 수행 과정

- 다음 그림과 같은 4개의 금속 분말이 있고, 배낭의 최대 용량은 40그램이다.



- Line 1~2의 결과
 - $S = [\text{백금}, \text{금}, \text{은}, \text{주석}]$

<u>물건</u>	<u>단위</u>	<u>그램당 가치</u>
백금		6만원
금		5만원
은		4천원
주석		1천원

FractionalKnapsack 알고리즘 수행 과정

- Line 3
 - $L=\emptyset, w=0, v=0$ 로 각각 초기화한다.
- Line 4
 - $S=[\text{백금}, \text{금}, \text{은}, \text{주석}]$ 로부터 **백금**을 가져온다.
- Line 5
 - while-루프의 조건 $((w+\text{백금의 무게}) \leq C) = ((0+10) < 40)$ 이 '참'이다.
- Line 6
 - 백금을 배낭 L 에 추가시킨다. 즉, $L=[\text{백금}]$ 이 된다.
- Line 7
 - $w = w(\text{백금의 무게}) = 0+10g = 10g$
- Line 8
 - $v = v(\text{백금의 가치}) = 0+60\text{만원} = 60\text{만원}$
- Line 9
 - S 에서 백금을 제거한다. $S=[\text{금}, \text{은}, \text{주석}]$
- Line 10
 - S 에서 **금**을 가져온다.

FractionalKnapsack 알고리즘 수행 과정

□ Line 5

- while-루프의 조건 $((w + \text{금의 무게}) \leq C) = ((10 + 15) < 40)$ 이 '참'이다.

□ Line 6

- 금을 배낭 L에 추가시킨다. $L = [\text{백금}, \text{금}]$

□ Line 7

- $w = w + (\text{금의 무게}) = 10g + 15g = 25g$

□ Line 8

- $v = v + (\text{금의 가치}) = 60\text{만원} + 75\text{만원} = 135\text{만원}$

□ Line 9

- S에서 금을 제거한다. $S = [\text{은}, \text{주석}]$

□ Line 10

- S에서 은을 가져온다.

□ Line 5

- while-루프의 조건 $((w + \text{은의 무게}) \leq C) = ((25 + 25) < 40)$ 이 '거짓'이므로 루프를 종료한다.

Fractional Knapsack 알고리즘 수행 과정

□ Line 11

- if-조건 $((C-w) > 0)$ 이 '참'이다. 즉, $40-25 = 15 > 0$ 이기 때문이다.

□ Line 12

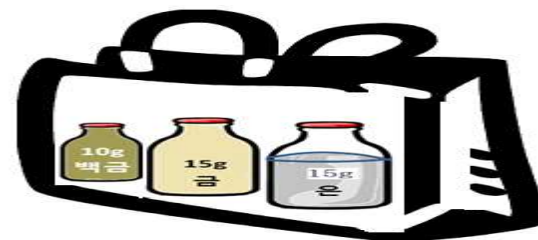
- 따라서 은을 $C-w=(40-25)=15\text{g}$ 만큼만 배낭 L에 추가시킨다.

□ Line 13

- $v = v + (15\text{g} \times 4\text{천원/g}) = 135\text{만원} + 6\text{만원} = 141\text{만원}$

□ Line 14

- 배낭 $L = [\text{백금 } 10\text{g}, \text{ 금 } 15\text{g}, \text{ 은 } 15\text{g}]$ 과 가치의 합 $v = 141\text{만원}$ 을 리턴한다.



시간복잡도

□ Line 1

- n 개의 물건 각각의 단위 무게 당 가치를 계산하는 데는 $O(n)$ 시간 걸린다.

□ Line 2

- 물건의 단위 무게 당 가치에 대해서 내림차순으로 정렬하기 위해 $O(n \log n)$ 시간이 걸린다.

□ Line 5~10

- while-루프의 수행은 n 번을 넘지 않으며, 루프 내부의 수행은 $O(1)$ 시간이 걸린다.

□ Line 11~14

- 각각 $O(1)$ 시간 걸린다.

□ 알고리즘의 시간복잡도는

$$O(n) + O(n \log n) + n \times O(1) + O(1) = O(n \log n) \text{이다.}$$

응용

- 0-1 배낭 문제는 최소의 비용으로 자원을 할당하는 문제로서, 조합론, 계산이론, 암호학, 응용수학 분야에서 기본적인 문제로 다루어진다.
- '버리는 부분 최소화시키는' 원자재 자르기
- 자산투자 및 금융 포트폴리오에서의 최선의 선택

2. 동전 거스름돈 문제

- 동전 거스름돈 (Coin Change) 문제를 해결하는 가장 간단하고 효율적인 방법
 - 남은 액수를 초과하지 않는 조건하에 '욕심내어' 가장 큰 액면의 동전을 취하는 것
- 동전 거스름돈 문제의 최소 동전 수를 찾는 그리디 알고리즘
 - 단, 동전의 액면은 500원, 100원, 50원, 10원, 1원

CoinChange 알고리즘

CoinChange

입력: 거스름돈 액수 W

출력: 거스름돈 액수에 대한 최소 동전 수

1. $change = W, n500 = n100 = n50 = n10 = n1 = 0$
// n500, n100, n50, n10, n1은 각각의 동전 카운트
2. while ($change \geq 500$)
 $change = change - 500, n500++$ // 500원짜리 동전 수를 1 증가
3. while ($change \geq 100$)
 $change = change - 100, n100++$ // 100원짜리 동전 수를 1 증가
4. while ($change \geq 50$)
 $change = change - 50, n50++$ // 50원짜리 동전 수를 1 증가
5. while ($change \geq 10$)
 $change = change - 10, n10++$ // 10원짜리 동전 수를 1 증가
6. while ($change \geq 1$)
 $change = change - 1, n1++$ // 1원짜리 동전 수를 1 증가
7. return ($n500 + n100 + n50 + n10 + n1$)
 // 총 동전 수를 리턴한다.

CoinChange 알고리즘

□ Line 1

- Change를 입력인 거스름돈 액수 W로 놓고, 각 동전 카운트를 $n_{500} = n_{100} = n_{50} = n_{10} = n_1 = 0$ 으로 초기화

□ Line 2~6

- 차례로 500원, 100원, 50원, 10원, 1원짜리 동전을 각각의 while 루프를 통해 현재 남은 거스름돈 액수인 chang를 넘지 않는 한 계속해서 같은 동전으로 거슬러 주고, 그 때마다 각각의 동전 카운트를 1 증가

□ Line 7

- 동전 카운트들의 합을 리턴

CoinChange 알고리즘

□ 그리디 알고리즘의 근시안적인 특성

- CoinChange 알고리즘은 남아있는 거스름돈인 chang에 대해 가장 높은 액면의 동전을 거스르며,
- 500원짜리 동전을 처리하는 line 2에서는 100원짜리, 50원짜리, 10원짜리, 1원짜리 동전을 몇 개씩 거슬러 주어야 할 것인지에 대해서는 전혀 고려하지 않는다.

CoinChange 알고리즘의 수행 과정

□ 거스름돈이 760원일 경우

□ Line 1

- $\text{change} = 760$
- $n_{500} = n_{100} = n_{50} = n_{10} = n_1 = 0$ 으로 초기화

□ Line 2

- change 가 500보다 크므로 while-조건이 '참'이어서 $\text{change} = \text{change} - 500 = 760 - 500 = 260$ 이 되고, $n_{500} = 1$
- change 가 500보다 작으므로 line 2의 while-루프는 더 이상 수행되지 않는다.

CoinChange 알고리즘의 수행 과정

□ Line 3

- $\text{change} > 100$ 이므로 while-조건이 '참'이 되어서 $\text{change} = \text{change} - 100 = 260 - 100 = 160$ 이 되고, $n_{100} = 1$ 이 된다.
- $\text{Change} > 100$ 이므로 while-조건이 역시 '참'이라서 $\text{change} = \text{change} - 100 = 160 - 100 = 60$ 이 되고, $n_{100} = 2$
- 그 다음엔 change 가 60이므로 100보다 작아서 while-루프는 수행되지 않는다.

□ Line 4

- $\text{Change} > 50$ 이므로 while-조건이 '참'이라서 $\text{change} = \text{change} - 50 = 60 - 50 = 10$ 이 되고, $n_{50} = 1$
- Change 가 50보다 작으므로 while-루프는 수행되지 않는다.

CoinChange 알고리즘의 수행 과정

□ Line 5

- $\text{change} \geq 10$ 이므로 while-조건이 '참'이라서 $\text{change} = \text{change} - 10 = 10 - 10 = 0$ 이 되고, $n10 = 1$ 이 된다.
- 그 다음엔 change 가 10보다 작으므로 while-루프는 수행되지 않는다.

□ Line 6

- $\text{change} = 0$ 이므로 while-조건이 '거짓'이 되어 while-루프는 수행되지 않는다.

□ Line 7

- $n500 + n100 + n50 + n10 + n1 = 1 + 2 + 1 + 1 + 0 = 5$ 를 리턴한다.

CoinChange 알고리즘의 수행 과정

- 그런데 만일 한국은행에서 160원짜리 동전을 추가로 발행한다면, CoinChange 알고리즘이 항상 최소 동전 수를 계산할 수 있을까?
- 거스름돈이 200원이라면, CoinChange 알고리즘은 160원짜리 동전 1개와 10원짜리 동전 4개로서 총 5개를 리턴한다.



CoinChange 알고리즘의 결과

CoinChange 알고리즘의 수행 과정

- 그러나 200원에 대한 최소 동전 수는 100원짜리 동전 2개



CoinChange 알고리즘의 결과



최소 동전의 거스름돈

- CoinChange 알고리즘은 항상 최적의 답을 주지 못함
 - 따라서 실제로는 거스름돈에 대한 그리디 알고리즘이 적용되도록 동전이 발행됨

동전 거스름돈 문제 구현

```
def CoinChange(w):
    change=w
    n500=n100=n50=n10=n1=0

    while change >= 500:
        change = change-500
        n500 += 1
    while change >= 100:
        change = change-100
        n100 += 1
    while change >= 50:
        change = change-50
        n50 += 1
    while change >= 10:
        change = change-10
        n10 += 1
    while change >= 1:
        change = change-1
        n1 += 1
    return (n500+n100+n50+n10+n1)
```

```
money = int(input("교환할 돈은 얼마?"))
change = CoinChange(money)
print(money,"에 대한 총 교환 동전수는", change,"개 입니다.")
```

동전 거스름돈 문제 구현

```
money = 0
money = int(input("교환할 돈은 얼마?"))

c50000 = money // 50000
money %= 50000 # 50000으로 나눈 나머지

c10000 = money // 10000
money %= 10000 # 10000으로 나눈 나머지

c5000 = money // 5000
money %= 5000 # 5000으로 나눈 나머지

c1000 = money // 1000
money %= 1000 # 1000으로 나눈 나머지

c500 = money // 500
money %= 500 # 500으로 나눈 나머지

c100 = money // 100
money %= 100 # 100으로 나눈 나머지

print('\n오만원\t==> %d 개'%c50000)
print('\n만원\t==> %d 개'%c10000)
print('\n오천원\t==> %d 개'%c5000)
print('\n천원\t==> %d 개'%c1000)
print('\n오백원\t==> %d 개'%c500)
print('\n백원\t==> %d 개'%c100)
print('\n잔돈\t==> %d 원'%money)
```



Thank You